25901637

MGU-BCA (HONOURS) REGULAR/IMPROVEMENT/REAPPEARANCE EXAMINATION APRIL 2026
SECOND SEMESTER
Core Course (CC) - MG2CCRBCA101 - DATA STRUCTURES

Part A

Answer all questions. Each question carries 2 marks

1. Differentiate between best-case and worst-case time complexity of an algorithm. [A] / [CO1]

Best case is the function which performs the minimum number of steps on input data of n elements. Worst case is the function which performs the maximum number of steps on input data of size n.

2. What is the role of the "top" variable in an array-based stack implementation? [U] / [CO2]

The top pointer points to the head of the list, which is the top of the stack. Elements are pushed by adding to the top and popped by removing the top node and adjusting the top pointer.

3. Explain the differences between singly linked lists and doubly linked lists. [An] / [CO3]

Singly linked lists contain one pointer per node, allowing only forward traversal and using less memory. Doubly linked lists contain two pointers (next and previous), allowing bidirectional traversal and efficient deletions at the cost of higher memory usage.

4. Describe a subgraph. How is it derived from a given graph [U] / [CO4]

A subgraph is a graph formed from a subset of the vertices and edges of a larger graph, known as the parent graph or original graph. It is essentially a smaller portion of the original graph that preserves the original relationships (connections) between the chosen vertices.

5. What is the difference between an input-restricted and an output-restricted Double-ended-queue? [An] / [CO2]

Input-restricted deques allow insertion at only one end while allowing deletion from both ends, whereas output-restricted deques allow deletion at only one end while allowing insertion at both ends. [2x5 = 10]

Part B

Answer any 5 questions .Each question carries 6 marks

6. Write an algorithm to find the largest element in an array and analyse its time complexity. [A] / [CO1]

Algorithm FindLargest(A, n)
Input: An array A of n numbers
1. max = A[0]
2. for i = 1 to n - 1 do
3. if A[i] > max then
4. max = A[i]
5. return max

Time Complexity Analysis :- Time Complexity, O(n), where n is the number of elements in the array.

To determine the time complexity, we count the number of basic operations (comparisons) relative to the input size n

1. Initialization: Setting max = A[0] takes constant time.
2. The Loop: The loop runs from the second element ((i=1)) to the last element ((i=n-1)). This means it executes (n-1) times.
3. The Comparison: Inside each iteration of the loop, exactly one comparison (A[i] > max) is performed.
4. Total Operations: Since there are (n-1) comparisons, the total work is proportional to (n)

7. Define postfix notation and explain how it simplifies the evaluation of arithmetic expressions[K] / [CO2]

Postfix notation, or Reverse Polish Notation (RPN), is a mathematical method where operators follow their operands (e.g., 5 2 + for (5+2)). It simplifies expression evaluation by eliminating the need for parentheses and operator precedence rules, allowing computers to process complex formulas efficiently using a simple stack data structure. -2 marks

Write algorithm to evaluate post fix expressions - 4 marks

8. Write an algorithm for deleting the first and last node in a circular linked list. Explain it using a suitable example. [A] / [CO3]

Deletion from the beginning of the circular linked list

Step-by-step approach:

- Check if List is Empty: If last is nullptr, print "List is empty" and return nullptr.
- Get Head Node: Set head to last->next.
- Check for Single Node: If head equals last, delete head and set last to nullptr.
- Handle Multiple Nodes:
  => Update last->next to point to head->next.
  => Delete head.
- Return Updated last

Deletion at the end of Circular linked list

Step-by-step approach:

- Check if the list is empty (last == None). If yes, return None.
- Set head = last.next.
- If the list has only one node (head == last), delete it by setting last = None and return.
- Traverse the list using a pointer curr starting from head until curr.next == last.
- Update curr.next = head to skip the last node.
- Delete the last node and update last = curr.
- Return the updated last.

Give examples

9. Briefly explain the different types of rotations in an AVL tree. [U] / [CO4]

AVL tree rotations are operations used to maintain a balanced binary search tree, ensuring the height difference between left and right subtrees (balance factor) is at most 1.

- Rotations: rotations are designed to restore balance in O(1) time while ensuring the overall time complexity remains O(log n). AVL Trees use four cases to rebalance themselves after insertions and deletions: Left-Left (LL), Right-Right (RR), Left-Right (LR) and Right-Left (RL)

10. Write a C program to implement a linear search algorithm. How does the dataset size affect its performance? [A] / [CO1]

Write program 5 marks

Impact of Dataset Size on Performance 1 mark

The relationship between dataset size and performance is defined by several key factors:

Linear Growth, Worst-Case Scenario, Scalability Issues, Minimal Overhead

11. Describe hashing and the different types of mathematical functions that can be used as hash functions. [U] / [CO3]

A hash function is a function that takes an input (or key) of arbitrary size and converts it into a fixed-size value, called a hash value or hash code.

Types of Mathematical Hash Functions

Division Method (Modulo), Multiplicative Hashing, Mid-Square Method, Folding Method, Cryptographic Hash Functions

12. Compare binary trees and binary search trees with examples [An] / [CO4]

Binary Tree is a non-linear and hierarchical data structure where each node has at most two children referred to as the left child and the right child. The topmost node in a binary tree is called the root, and the bottom-most nodes (having no children) are called leaves.

A binary search tree is a hierarchical data structure where each node has at most two children, with values ordered such that left child values are smaller and right child values are greater.

A binary Search Tree is a node-based binary tree data structure that has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.
- There must be no duplicate nodes.

Give examples [6x5 = 30]

Part C

Answer any 2 questions .Each question carries 15 marks

13. Explain how a two-dimensional array is represented in memory. Write a C program to multiply two matrices. [A] / [CO1]

A two-dimensional array is stored in memory as a contiguous block of sequential memory locations, even though it is conceptually viewed as a table with rows and columns.

Memory Representation- 6 marks

There are two primary methods for organizing these elements in linear memory:

- Row-Major Order: Elements are stored row by row. All elements of the first row are placed first, followed by all elements of the second row, and so on. This is the standard method used in C and C++.
- Column-Major Order: Elements are stored column by column. All elements of the first column are placed first, then the second column, etc..

Write Program - 9 marks

14. Compare simple queues and circular queues with examples and their applications.] [An] / [CO2]

Simple queues and circular queues both follow the FIFO (First-In-First-Out) principle, but differ in memory management. Simple queues are linear, causing wasted space at the front when items are deleted. Circular queues connect the last position back to the first, enabling efficient reuse of empty space

Simple Queue Applications

Printer Queues, Message Queues, Breadth-First Search (BFS), Call Centre Management

Circular Queue Applications

CPU Scheduling, Memory Management, Traffic Systems, Round-Robin Token Generation.

Give examples

15. Explain the different types of recursion? Write a C program to generate fibonacci series upto a limit using recursion. [A] / [CO3]

Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem

Types of Recursion

Recursion can be categorized based on how and when the recursive call occurs:

- Direct Recursion**:** The most common form, where a function directly calls itself within its own body.
- Indirect Recursion**:** Occurs when a function calls another function, which then eventually calls the first function, creating a circular chain.

Program [15x2 = 30]

Prepared by
Sahala K I
Assistant Proessor,
Jai Bharat Arts & Sciience College, Arackappady
Ph; 984732712